

XGBoost

1. Why So Many ML Algorithms?

- Every algorithm is data-specific or scenario-specific
- Old algorithms (70s–80s): worked only on specific types of data
 - Linear Regression → only linear data
 - Naive Bayes → only text data
- 90s algorithms: more powerful & general
 - Random Forest, SVM, Gradient Boosting

Problems with 90s Algorithms

- Still had overfitting issues
- Slow on big data (scalability problem)

 *Solution → XGBoost (2014)*

2. What is XGBoost?

XGBoost = Extreme Gradient Boosting

- Created by Tianqi Chen in 2014
- It is NOT a new algorithm — it is a library built on top of Gradient Boosting
- Takes Gradient Boosting + adds powerful software optimizations

XGBoost combines two things:

-  Machine Learning — from Gradient Boosting
-  Software Engineering — speed optimizations

3. Brief History of XGBoost

Stage	Year	What Happened
Created	2014	Tianqi Chen published the XGBoost paper
Kaggle Buzz	2016	16 out of 29 winning Kaggle solutions used XGBoost
Open Source	After 2016	Code made public → global contributions → rapid growth

- Tianqi Chen himself won the Higgs Boson competition on Kaggle using XGBoost
- Growth that would take 10 years happened in just 5 years

4. Why Gradient Boosting Was Chosen as the Base?

- ✓ Flexible — works with any differentiable loss function
- ✓ Great Performance — good results on most datasets
- ✓ Robust — stable results with proper regularization
- ✓ Versatile — works for regression, classification, ranking

Only problem → slow on big data + slight overfitting. XGBoost fixed exactly these two problems.

5. Three Main Goals of XGBoost

Goal	Description
1. Performance	Better accuracy, less overfitting
2. Speed	Handle big data fast
3. Flexibility	Usable by everyone, everywhere

6. Flexibility Features

6.1 Cross-Platform

- Runs on Windows, Mac, Linux — no restrictions

6.2 Multiple Programming Languages

- Supports: Python, R, Java, Scala, Julia, C, C++, Ruby, Swift
- You can build a model in Python and use it in Java directly
- Huge advantage for enterprise applications

6.3 Compatible with Popular Libraries

Category	Libraries
Data Analysis	NumPy, Pandas, Matplotlib
ML Framework	Scikit-learn
Distributed Computing	Spark, PySpark, Dask
Model Explainability	SHAP, LIME
Deployment	Docker, Kubernetes
Workflow	MLflow, Airflow

6.4 Supports Many Problem Types

-  Regression
-  Binary Classification
-  Multi-class Classification
-  Time Series Forecasting
-  Ranking Problems
-  Custom Loss Functions — define your own!

7. Speed Optimizations (6 Key Techniques)

Proof: On 10,000 rows × 200 columns dataset: Gradient Boosting → 72 seconds XGBoost → 6 seconds (12× faster!)

1. Parallel Processing

- Boosting is sequential (model 1 → model 2 → model 3...)
- But within each tree, finding splits runs in parallel
- Example: 200 features → all tested simultaneously across CPU cores
- Hyperparameter: set `n_jobs = -1` to activate

2. Column Block Data Structure

- Normal algorithms store data row-wise
- XGBoost stores data in column blocks (column-wise)
- This makes parallel processing on features much easier and faster

⚡ 3. Cache Awareness

- XGBoost uses cache memory — tiny, super-fast memory inside CPU
- Stores frequently used values (like histogram bin values) in cache
- Avoids repeated trips to RAM → speeds up tree building

💡 *Analogy: Keep frequently used cooking ingredients next to you instead of going to the fridge every time*

⚡ 4. Out-of-Core Computing

- Problem: Dataset (10 GB) > RAM (8 GB) → cannot load it all!
- Solution: Split dataset into chunks → load one chunk at a time
- Train → next chunk → train → repeat

```
|tree_method = "hist" ← activate out-of-core computing
```

⚡ 5. Distributed Computing

- Use multiple machines (nodes) to train simultaneously
- Split data across nodes → each trains on its portion → results combined by a master node
- Much faster than out-of-core (parallel across machines, not just chunks)
- Uses: Dask or Kubernetes

	Out-of-Core	Distributed
Machines	1	Multiple
Processing	Sequential chunks	Parallel across nodes
Speed	Good	Better

⚡ 6. GPU Support

- All heavy tasks (histogram creation, split finding) are highly parallelizable
- XGBoost supports GPU training natively
- GPUs have thousands of small cores → perfect for parallel tasks

```
|tree_method = "gpu_hist" ← activate GPU training
```

💡 *This is why it is called "Extreme" — they took software optimization to the extreme!*

8. Performance Optimizations (5 ML Techniques)

🔧 1. Regularized Learning Objective

- Normal Gradient Boosting: regularization was separate (learning rate, pruning)
- XGBoost: regularization term is built INTO the loss function by default

Formula: Loss = Differentiable Loss + Regularization Term ($\lambda \times \text{leaf weights}$)

- Benefit: Model automatically reduces overfitting while minimizing loss

🔧 2. Handling Missing Values

- Other algorithms → crash on missing values (need preprocessing)
- XGBoost → handles missing values automatically

How it works:

1. For each split, tries sending missing values LEFT → calculates gain
2. Tries sending missing values RIGHT → calculates gain
3. Picks whichever direction gives maximum gain

✅ *No need for manual imputation — XGBoost handles it internally!*

🔧 3. Approximate Tree Learning + Weighted Quantile Sketch

- Problem with Greedy Search: tries every possible split → accurate but very slow
- Solution: Approximate Tree Learning → bin the continuous values into groups

How bins are created — Weighted Quantile Sketch:

- Study the data distribution first
- Where data is sparse → make big bins
- Where data is dense → make small bins
- Much smarter than uniform binning (0–10, 10–20...)

💡 *This is called Histogram-Based Training — faster speed + maintained accuracy*

4. Tree Pruning with Gamma

- XGBoost offers both pre-pruning and post-pruning
- Special hyperparameter: Gamma (γ)
 - A new branch is only added if it significantly reduces the loss
 - If $\text{gain} < \text{gamma} \rightarrow$ branch is NOT created
- Reduces complexity and overfitting automatically

5. Key Hyperparameters

Hyperparameter	Purpose
max_depth	Control tree depth
learning_rate	Control contribution of each tree
n_estimators	Number of trees
gamma	Minimum loss reduction for a split
lambda	L2 regularization term
tree_method	"hist" (out-of-core) "gpu_hist" (GPU)
n_jobs	Number of CPU cores for parallel processing (-1 = all)

9. XGBoost vs Other Gradient Boosting Libraries

Library	Creator	Strength
XGBoost	Tianqi Chen	Best all-around performance & speed
LightGBM	Microsoft Research	Faster training, lower memory usage
CatBoost	Yandex	Best for categorical features

💡 All three are powerful — worth learning all of them!

10. Quick Summary

XGBoost = Gradient Boosting + Extreme Software Optimization

Why XGBoost is Great:

-  10–12× faster than Gradient Boosting
-  Handles missing values automatically
-  Built-in regularization (less overfitting)
-  Works with any size dataset (out-of-core + distributed)
-  GPU support for ultra-fast training
-  Supports multiple languages & popular libraries
-  Go-to tool for Kaggle competitions & industry projects

 *Must-Know for: Data Scientists, ML Engineers & Kaggle competitors*

Supervised Learning

XGBoost

(Extreme Gradient Boosting)

XGBoost is an optimized implementation of Gradient Boosting that builds an ensemble of decision trees sequentially, where each new tree corrects the errors of the previous trees.

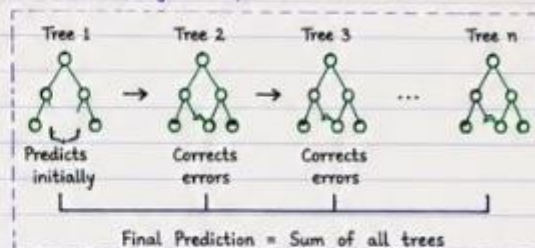
Why XGBoost?

- ✓ High Accuracy
- ✓ Handles Missing Values
- ✓ Regularization (Less Overfitting)
- ✓ Fast and Scalable
- ✓ Works for Regression & Classification

* How it Works (Short)

- ① Start with a simple model (constant prediction)
- ② Calculate error (residuals)
- ③ Build a decision tree to predict the residuals
- ④ Add the tree to the model with a learning rate (η)
- ⑤ Repeat steps 2-4 for many trees
- ⑥ Final prediction is the sum of all trees

* Model Building (Concept)



* The Math Behind XGBoost

XGBoost minimizes the following regularized objective function:

$$\text{Obj}(\theta) = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

Where,

- $\ell(y_i, \hat{y}_i)$ = loss function (e.g., squared error, log loss)
- $\hat{y}_i$ = predicted value
- f_k = k^{th} tree
- $\Omega(f_k)$ = regularization term to control complexity

Using Taylor Expansion (up to 2nd order), the objective at iteration t becomes:

$$\text{Obj}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Where,

- g_i = first derivative (gradient) = $\frac{\partial}{\partial \hat{y}_i^{(t-1)}} \ell(y_i, \hat{y}_i^{(t-1)})$
- h_i = second derivative (hessian) = $\frac{\partial^2}{\partial \hat{y}_i^{(t-1)^2}} \ell(y_i, \hat{y}_i^{(t-1)})$

XGBoost finds the best tree structure and leaf weights to minimize this objective.

* Regularization Term $\Omega(f_k)$

$$\Omega(f_k) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Where,

- T = number of leaves
- w_j = score (weight) of leaf j
- γ = penalty for more leaves
- λ = L2 regularization on leaf weights

In Short: XGBoost = Boosting + Decision Trees + Regularization + Speed
Better accuracy, less overfitting, handles real-world data efficiently.

* Key Hyperparameters

- eta (learning rate) - Shrinks the contribution of each tree
- n_estimators - Number of boosting rounds (trees)
- max_depth - Maximum depth of each tree
- subsample - Fraction of data used per tree
- colsample_bytree - Fraction of features used per tree
- lambda - L2 regularization term
- alpha - L1 regularization term
- gamma - Minimum loss reduction for a split

* Practical Example

Problem: Predict House Prices

Data: Features like area, location, number of rooms, age of house, etc.

Target: Price of the house

How XGBoost helps?

- It learns complex patterns between features and price by combining many weak trees.
- Handles missing values automatically.
- Gives higher accuracy than single decision tree.

Area (sq ft)	Rooms	Location Score	Price (Target)
1200	3	8.5	50 Lakh
1500	4	9.0	65 Lakh
...	...	...	...

XGBoost will learn from this data and predict the price of a new house.

* Why it Works So Well?

- Sequential learning of errors ✓
- Regularization avoids overfitting ✓
- Efficient handling of large datasets ✓
- Flexible - works for many problems ✓

